Where the Internet Ends: A Serverless SMS Gateway for Large Language Model Access in Resource-Constrained Regions

Meet Rajani[1], Kirtan Ugreja[2], and Jalpesh Vasa[3]

[1] Department of Information Technology, CSPIT, CHARUSAT, India
rajanimeet2005@gmail.com
[2] Department of Computer Engineering, DEPSTAR, CHARUSAT, India
ugrejakirtan0@gmail.com
[3] Department of Artificial Intelligence and Machine Learning, CSPIT, CHARUSAT, India
jalpeshvasa.it@charusat.ac.in

Abstract. Billions of people in rural and low-resource regions remain cut off from modern AI—not because the technology cannot help them, but because it was never built to reach them. LLM-powered conversational systems assume internet-connected smartphones that much of the world does not have, while existing SMS-based alternatives rely on rigid rule-based logic incapable of handling real user needs. This paper proposes a serverless, SMS-native architecture that enables users to interact with a large language model through standard text messages on any mobile phone, without internet access. Incoming messages are routed through a commercial SMS gateway to an AWS Lambda function, which manages session context and queries Anthropic's Claude via Amazon Bedrock before returning a response—typically in under seven seconds on a 2G network. A multi-deployment framework allows a single backend to power independent domain-specific helplines, each configured with its own system prompt, language, and retention policy. A lightweight session memory mechanism, combining DynamoDB TTL expiry with LLM-generated summarisation, provides contextual continuity for returning users without storing full message histories. Evaluation across three prototype deployments demonstrates per-interaction costs of USD 0.010–0.020, mean response relevance scores above 4.0 on a five-point scale, and a 31% reduction in input token consumption through summarisation without significant loss in response quality. The system demonstrates that conversational AI can be delivered over the infrastructure the world already has—not the infrastructure it is still waiting for.

Keywords: SMS chatbot · large language models · rural AI access · serverless architecture · multi-deployment framework · session memory · low-connectivity environments · digital divide

1 Introduction

The promise of artificial intelligence is not being shared equally. Across much of the world, AI-powered tools are reshaping how people access healthcare information, learn new skills, navigate government services, and manage their livelihoods. Yet for the billions of people living in rural and low-resource regions—where internet connectivity is unreliable, mobile data is expensive, and smartphones remain out of reach—these advances might as well not exist. The digital divide is not merely a gap in device ownership; it is increasingly a gap in who gets to benefit from intelligent technology at all [**?**].

Mobile phones, however, tell a different story. Even in areas where internet infrastructure is sparse, basic mobile handsets are commonplace. Short Message Service (SMS)—the humble text message—runs on these devices without a data connection, functions on 2G and older cellular networks, and costs a fraction of a data-based interaction. For hundreds of millions of people, SMS is not a legacy technology; it is the primary way they communicate digitally [**?**]. This makes it a uniquely practical channel for reaching communities that more sophisticated platforms routinely overlook.

At the same time, large language models (LLMs) have matured to a point where they can hold coherent multi-turn conversations, answer complex domain-specific questions, and adapt their responses to user needs in real time [**?**]. The potential applications in underserved contexts are considerable—a farmer asking about crop disease, a patient describing symptoms in the absence of a nearby clinic, or a family navigating a disaster relief process. What stands in the way is not the capability of the technology, but its delivery.

This paper asks a straightforward question: can the conversational power of a modern large language model be delivered through a plain text message? We propose that it can. The system described in this work routes incoming SMS messages through a serverless cloud backend, manages conversational state, and queries an LLM to generate contextually relevant replies—all of which are returned to the user as standard text messages. No smartphone, application, or internet connection is required on the user's device.

Building a conversational system around SMS introduces practical constraints. SMS messages are stateless, meaning each message arrives independently without context. Storing complete conversation histories for every user is inefficient and expensive at scale. To address this challenge, the system introduces a session mechanism that stores active conversation context temporarily and later summarizes the interaction using the LLM itself. This summary is retained and used as context when the user returns, enabling continuity without storing full histories.

The architecture is designed to support multiple real-world deployments. A single backend can power several independent helplines—such as agricultural advisory services, rural healthcare assistance, or disaster response information systems—each configured with different prompts, policies, and languages.

The contributions of this paper are summarized as follows:

- A scalable serverless architecture enabling LLM-powered conversations over SMS.
- A lightweight session memory mechanism using TTL-based expiration and summarization.
- A multi-deployment framework supporting several domain-specific services within a single system.
- An empirical evaluation demonstrating feasibility in terms of latency, cost, and response quality.

The remainder of the paper is structured as follows. Section 2 discusses related work. Section 3 presents the system architecture. Section 4 explains the session and memory design. Section 5 describes the multi-deployment framework. Section 6 evaluates the system. Section 7 outlines limitations and future work, and Section 8 concludes the paper.

2 Related Work

Efforts to bring digital services to low-resource and rural communities have followed several threads over the past two decades—from early SMS-based information systems to mobile advisory platforms to, more recently, AI-powered conversational agents. Each of these approaches has advanced the field in meaningful ways, yet none has fully bridged the gap between the conversational power of modern large language models and the infrastructural realities of low-connectivity environments. This section reviews the most relevant prior work and identifies the limitations that motivate the approach proposed in this paper.

2.1 SMS-Based Information Systems

Long before smartphones became widespread, SMS was already enabling important digital services. Governments, NGOs, and researchers recognized early on that nearly any mobile phone could send and receive text messages, making SMS one of the few digital channels capable of reliably reaching rural populations. This led to the development of SMS-based platforms for agriculture, healthcare, and public administration—systems that delivered weather alerts to farmers, reminded patients about medication schedules, and distributed government announcements without requiring internet connectivity on the recipient's device [**?**,**?**].

These systems demonstrated real-world value across deployments in Sub-Saharan Africa, South Asia, and Southeast Asia. However, their interaction model remained limited. Most platforms relied on predefined commands or keyword-based queries, requiring users to learn specific formats when requesting information. For example, a farmer might send a message such as "MAIZE PRICE" to receive current market data, but could not ask a natural question such as "My maize leaves are turning yellow—what should I do?" and receive a contextual response. This rigidity significantly limited the usefulness of such systems when users faced complex or unexpected situations [**?**].

2.2 Chatbots for Rural and Development Contexts

As chatbot technology evolved, researchers began exploring its potential for development and humanitarian applications. Several systems were designed to allow farmers, patients, and citizens to submit open-ended questions through SMS or messaging platforms, with responses generated either through simple natural language processing pipelines or by human moderators reviewing incoming messages [**?**,**?**].

These hybrid approaches improved usability by removing strict command syntax and enabling more flexible communication. However, reliance on human moderators introduced scalability challenges. Systems requiring manual responses become expensive to operate, introduce delays, and depend heavily on sustained organizational support. Fully automated systems developed during this period typically relied on shallow intent classification and templated responses, limiting their ability to handle complex queries [**?**].

In humanitarian contexts, similar conversational systems have been deployed to support disaster response, healthcare access, and public information services. Many of these systems, however, depend on internet-based messaging platforms such as WhatsApp or dedicated mobile applications, which remain inaccessible to users with basic feature phones or limited connectivity [**?**].

2.3 Large Language Models in Conversational AI

The emergence of large language models (LLMs) has significantly expanded the capabilities of conversational AI systems. Models trained on large corpora of human language can generate coherent and contextually appropriate responses across a wide range of domains without relying on rigid decision trees or predefined templates [**?**]. This capability has important implications for open-ended advisory tasks such as agricultural guidance, medical information, and disaster response.

Despite this progress, most LLM-based systems remain tied to internet-connected environments. Current implementations are typically designed for web interfaces, mobile applications, or API integrations that assume reliable broadband or mobile data connectivity. These requirements make deployment difficult in environments where connectivity is intermittent or unavailable. As a result, the populations that could benefit most from AI-driven advisory services often remain excluded [**?**].

Recent research has explored lightweight or compressed LLM deployments intended for edge devices, aiming to reduce reliance on constant connectivity [**?**]. However, these approaches still generally require smartphones or local computing hardware and have not been adapted to feature phone environments where SMS remains the dominant communication channel.

2.4 Research Gap

The literature highlights a clear gap between the reach of SMS-based systems and the capabilities of modern AI technologies. SMS platforms demonstrate resilience

and accessibility in low-resource environments but are constrained by rule-based interaction models that cannot accommodate complex user queries. In contrast, LLM-based conversational systems provide flexible and context-aware responses but assume infrastructure that many rural communities do not possess.

Additionally, most existing solutions are designed for single-domain deployments. Systems created for agricultural advisory, for example, are rarely adaptable to other applications such as disaster response or public health information without substantial redesign. Furthermore, conversational context management remains a challenge in SMS environments, where messages are typically processed independently without mechanisms for maintaining user history across sessions.

The system proposed in this paper addresses these limitations by integrating an LLM inference backend with an SMS gateway through a scalable serverless architecture. It introduces a lightweight session memory mechanism based on summarization and supports a multi-deployment framework capable of serving multiple domain-specific services within a single infrastructure.

3 System Architecture

The architecture proposed in this paper is built around a simple but important idea: that the intelligence of a modern large language model should be reachable from the most basic mobile phone, over the most basic network, without asking anything more of the user than the ability to send a text message. Realising this in practice requires careful orchestration across telecommunications infrastructure, serverless cloud computing, AI inference services, and persistent storage. This section describes how these components are organised, how they interact, and why specific design decisions were made.

At a high level, the system consists of four principal layers: an SMS gateway, a serverless processing backend, an AI inference layer, and a database for session and profile management. Fig. 3 provides an overview of the end-to-end architecture and the flow of a message through the system.

3.1 SMS Gateway Layer

Every interaction begins with a text message. When a user sends an SMS to the service's registered phone number, the message is received by a commercial SMS gateway provider — in this implementation, one of Twilio, MSG91, or Gupshup, depending on the deployment region. The gateway's role is straightforward but essential: it translates the incoming SMS into an HTTP POST request and forwards it to the system's webhook endpoint. On the return path, it receives the AI-generated response from the backend and delivers it to the user's handset as a standard text message.

Relying on a commercial gateway provider has practical advantages beyond mere convenience. These providers manage carrier relationships, handle regional routing, support bulk message delivery, and maintain compliance with local

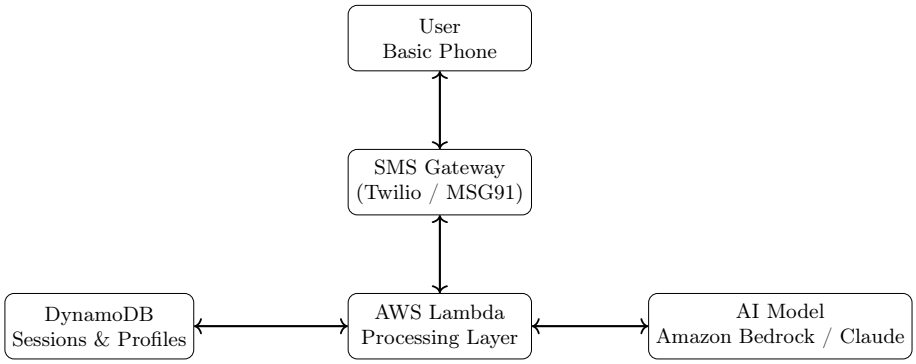

Fig. 1. Architecture of the Rural AI SMS Chatbot System

telecommunications regulations — complexities that would otherwise require significant dedicated infrastructure. For an architecture intended to be deployable by NGOs, government agencies, or small development teams, offloading these concerns to a managed gateway service is a deliberate and important design choice.

3.2 Serverless Processing Layer

The webhook endpoint is served by a function deployed on AWS Lambda. When triggered by an incoming SMS, the Lambda function is responsible for coordinating the entire processing pipeline: parsing the request, retrieving session data, constructing the model prompt, invoking the AI service, updating the session, and dispatching the reply.

Serverless infrastructure suits this application particularly well. Message traffic in public service deployments is inherently unpredictable — a disaster response helpline may receive thousands of messages within minutes of an event, then go quiet for hours. A traditional server-based architecture would require provisioning for peak load at all times, incurring unnecessary cost during idle periods. With Lambda, compute resources are allocated only when a message arrives and released immediately after processing, making the system both cost-efficient and naturally elastic.

The function also extracts the deployment identifier from the incoming request, which determines which service configuration — system prompt, language, retention policy — should be applied to the interaction. This identifier is what allows a single backend to serve multiple independent deployments simultaneously.

3.3 AI Inference Layer

Once the session context has been assembled, it is forwarded to a large language model via Amazon Bedrock — specifically, Anthropic's Claude. The model receives a structured prompt containing the deployment's system instructions, a

summary of the user's prior interactions (if any), the recent conversation history from the active session, and the current user message. From this input, it generates a natural language response appropriate to the domain and the user's query.

The decision to use a managed AI inference service rather than a self-hosted model reflects a deliberate trade-off. Hosting and maintaining LLM infrastructure introduces significant operational complexity and cost, particularly for organisations without dedicated machine learning engineering teams. Amazon Bedrock provides on-demand access to capable frontier models with no infrastructure management overhead, making it far more practical for the kinds of public service deployments this system targets.

3.4 Data Storage and Session Management

The system uses Amazon DynamoDB as its primary data store, organised into two tables that serve distinct purposes.

The **user_sessions** table stores the active conversation history for ongoing interactions. Each record is keyed on the user's phone number and deployment identifier and contains the full message exchange from the current session. A TTL attribute is set on every record, causing DynamoDB to automatically expire and delete sessions that have been inactive beyond a configurable threshold — typically one hour. This TTL mechanism is not merely a housekeeping convenience; it is a core part of the memory architecture, triggering the summarisation process described later and ensuring that full conversation histories are never retained indefinitely.

The **user_profiles** table stores persistent, cross-session information. When a session ends — either through an explicit user command or TTL expiry — a summary of the conversation is written to this table alongside metadata such as the user's preferred language and session count. When the same user returns in a future session, this summary is retrieved and injected into the model's context, providing continuity without the cost of replaying the entire conversation history.

3.5 Message Processing Pipeline

The journey of a single user message through the system follows a structured pipeline (Fig. **??**):

Step 1 — Message Reception. The user sends a text message from any SMS-capable phone to the service number. The SMS gateway receives the message, converts it to an HTTP request, and forwards it to the Lambda webhook endpoint.

Step 2 — Request Parsing. The Lambda function extracts key metadata from the incoming request: the sender's phone number, the message content, the timestamp, and the deployment identifier.

Step 3 — Session Retrieval. The backend queries DynamoDB for an active session associated with the user's phone number and deployment identifier.

If a session exists, its conversation history is retrieved; otherwise, a new session record is initialised.

Step 4 — Context Construction. The system assembles the prompt sent to the language model. This includes the deployment's system prompt, the summarised history from the user's previous session (if available), recent messages from the current session, and the new incoming message.

Step 5 — AI Response Generation. The assembled prompt is sent to the language model via Amazon Bedrock, which generates a natural language response tailored to the user query.

Step 6 — Session Update. The exchange is appended to the session record in DynamoDB and the last-active timestamp is refreshed.

Step 7 — SMS Delivery. The response is returned to the SMS gateway and delivered to the user's handset. If necessary, long responses are split across multiple SMS segments.

3.6 Multi-Deployment Framework

A defining feature of this architecture is that it is not limited to a single use case. Each deployment is identified by a unique deployment identifier mapped to a registered phone number and a configuration record specifying the system prompt, language preferences, session TTL, and response policies.

Multiple deployments can operate simultaneously on the same backend infrastructure. This allows organisations to run services such as agricultural advisory systems, healthcare information platforms, or disaster response helplines without building separate systems for each domain.

3.7 Fault Tolerance and Reliability

Reliability is essential for public service deployments. The architecture incorporates several mechanisms to ensure resilience. AWS Lambda scales automatically to handle traffic spikes, DynamoDB provides built-in redundancy and failover capabilities, and SMS gateway providers queue undelivered messages and retry delivery when connectivity improves.

Together, these features allow the system to remain operational even under unstable network conditions.

3.8 Cost-Efficient Design

Cost efficiency is a core design goal. The organisations most likely to deploy this system — rural health NGOs, disaster response agencies, and government programmes — often operate under tight financial constraints.

Serverless infrastructure ensures compute resources are used only when processing messages. The session summarisation mechanism reduces token usage during AI inference, lowering operational costs. Finally, responses are intentionally concise, minimising the number of SMS segments required per interaction and reducing messaging fees.

4 Session Memory and Context Management

Conversation is inherently cumulative. What a user asks in a later message is often shaped by what was discussed earlier. In web-based chat systems this continuity is easy to maintain because connections are persistent and storage is inexpensive. SMS-based interaction, however, operates under very different constraints. Messages arrive independently, have strict size limits, and each additional token sent to a language model increases inference cost. Creating a coherent conversational experience therefore requires a memory architecture that is efficient, lightweight, and carefully designed.

The mechanism proposed in this work combines active session storage for ongoing interactions with language-model-generated summarisation for long-term continuity. Together, these components allow the system to maintain context during a session while preserving essential information across sessions without storing complete histories indefinitely.

4.1 Active Session Handling

When a message arrives, the backend first determines whether the sender already has an active session. Sessions are indexed using two identifiers: the user's phone number and the deployment identifier of the service being accessed. This allows the same user to interact with multiple deployments independently without mixing context between them.

If an active session exists, its stored conversation history is retrieved and included in the prompt sent to the language model. If no session exists, the system initializes a new session and begins the interaction from a clean state.

To prevent the session record from growing indefinitely, only a sliding window of recent exchanges is retained. Older messages are progressively trimmed as the conversation continues. This ensures that the context remains relevant while keeping the token count manageable for model inference.

4.2 Session Expiration via TTL

SMS conversations are typically intermittent. A user may send a message, receive a response, and then return hours or days later. Maintaining inactive sessions indefinitely would create unnecessary storage overhead and provide little benefit.

Each session stored in DynamoDB therefore includes a TTL (time-to-live) attribute assigned during creation or update. If no additional message is received within a predefined inactivity period—one hour in the prototype implementation—the record is automatically marked for expiration.

The expiration event is captured through a DynamoDB Stream, which triggers a dedicated AWS Lambda function responsible for summarising the session before deletion. This event-driven design eliminates the need for scheduled cleanup tasks or manual management.

4.3 Conversation Summarisation

Before an expired session is removed, the system sends the conversation history to the language model with instructions to generate a concise summary of the exchange. The summary captures the user's main concerns, the assistance provided, and any relevant preferences or circumstances mentioned during the conversation.

The resulting summary is then stored in the *user_profiles* table while the detailed message history is discarded.

Using the language model itself for summarisation allows the system to preserve the most meaningful aspects of the interaction rather than simply truncating text. Important contextual elements—such as a symptom described by a patient or a crop issue reported by a farmer—are retained in a compact form suitable for future interactions.

4.4 Returning User Context

When a user initiates a new session after a previous one has expired, the system checks the *user_profiles* table for an existing summary. If available, this summary is injected into the model's prompt as contextual background before processing the user's new message.

This allows the system to maintain continuity between sessions. For example, a farmer who previously asked about pest management does not need to repeat their entire situation in the next conversation. Similarly, a returning user seeking follow-up guidance can continue the discussion more naturally.

Importantly, the system stores only the most recent summary rather than building extensive long-term profiles. Apart from the phone number used as a database key, no additional personal information is retained. This design supports both privacy considerations and storage efficiency.

4.5 Design Rationale and Advantages

The session memory architecture reflects a deliberate balance between contextual continuity and operational cost. Retaining complete conversation histories would increase model context and storage usage over time, eventually becoming impractical at scale. Summarisation provides a bounded memory footprint while preserving the most relevant information.

Another advantage of this design is its deployment independence. The same session management mechanism functions across all services built on the platform, whether they relate to agriculture, healthcare, or disaster response. This consistency simplifies system maintenance and enables rapid creation of new deployments.

Finally, because summarisation is triggered asynchronously after session expiration, it does not introduce additional latency into the active messaging pipeline. Users experience responsive interactions while memory management occurs in the background.

Table 1. Session Memory Design Properties

Feature	Description
Active Session Storage	Sliding window of recent messages
Session Expiration	DynamoDB TTL (1 hour inactivity)
Long-Term Memory	LLM-generated summary per user
Privacy Approach	No full history retention
Scalability	Event-driven summarisation via Lambda

5 Multi-Model Integration and Deployment Framework

No single AI model is equally suited to every task. A model optimised for open-ended dialogue may not be the best choice for structured medical triage; a domain-specific model fine-tuned on agricultural data may outperform a general-purpose LLM when answering questions about crop disease but offer little advantage for navigating government benefit schemes. In a platform designed to serve diverse communities across radically different domains, hardcoding a dependency on any one model would impose an unnecessary ceiling on what the system can do and for whom.

The framework described in this section removes that limitation. Rather than routing every user request to a single language model, the system incorporates a model integration layer that dynamically selects the most appropriate model or external AI service for each interaction, based on the deployment context and the nature of the user's query.

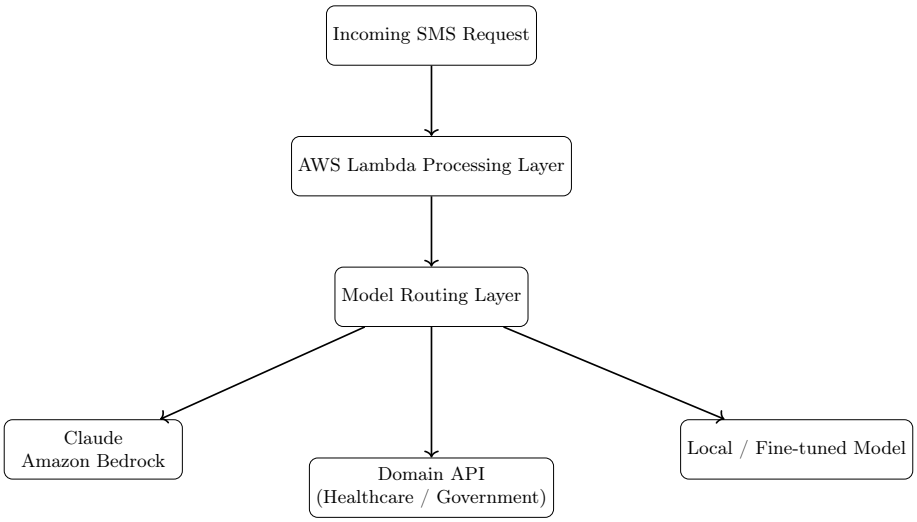

Fig. 2. Multi-model routing framework enabling deployment-specific AI model selection.

5.1 Model Routing Architecture

At the core of the framework is a routing layer positioned between the serverless processing backend and the available AI services. When a user message arrives and its session context has been assembled, the routing layer evaluates the deployment identifier and, where necessary, the category of the request to determine which model should handle the query.

Routing behaviour is defined through deployment configuration. Each service specifies a primary model, optional fallback models, and content-based routing rules that can direct specialised queries to appropriate services. Because this configuration is stored externally alongside system prompts and language preferences, operators can update routing behaviour without modifying the backend code.

The routing layer communicates with each provider through a standardised interface that abstracts provider-specific request formats, authentication mechanisms, and response structures. This design allows new models or services to be integrated simply by implementing an adapter while leaving the rest of the system unchanged.

5.2 Supported Model and Service Types

The framework accommodates three primary categories of AI services.

Managed cloud language models. These models provide the default conversational capability for most deployments. In the prototype implementation, Anthropic's Claude model accessed through Amazon Bedrock serves this role. Managed models offer strong language understanding without requiring organisations to operate complex machine learning infrastructure.

Domain-specific external APIs. Certain queries benefit from specialised knowledge sources. For example, healthcare deployments may integrate clinical advisory APIs, while agricultural deployments may connect to crop disease diagnostic services. By delegating specialised queries to domain APIs, the platform extends its capabilities while maintaining safety and accuracy in regulated contexts.

Open-source or fine-tuned models. Some organisations prefer to host their own models due to cost considerations, data governance policies, or the availability of domain-specific training data. The framework supports locally hosted or private-cloud models, enabling deployments that require greater control over data handling and inference behaviour.

5.3 Deployment Configuration and Isolation

Each service built on the platform operates as an independent deployment with its own configuration, including model routing rules, system prompts, supported languages, and data retention policies. Although deployments share the same backend infrastructure, they remain logically isolated from one another. A query sent to one service cannot access or influence the context of another.

Creating a new deployment requires only the definition of a configuration record and the registration of a phone number with the SMS gateway provider. Because the architecture treats deployment behaviour as configuration rather than code, the platform can scale to support many independent services without requiring ongoing engineering effort for each one.

5.4 Model Management and Operational Monitoring

Operating across multiple AI providers requires coordination beyond simple request forwarding. The framework therefore includes management components responsible for authentication, credential handling, request formatting, response normalisation, and fallback logic in the event that a provider becomes unavailable.

These components operate transparently within the Lambda processing pipeline and introduce minimal additional latency. Performance metrics such as request latency, error rates, and service availability are logged for monitoring and evaluation. This telemetry also supports the performance analysis presented later in the paper.

5.5 Rationale for a Multi-Model Approach

Designing the architecture around model flexibility reflects the realities of deploying AI-powered public services. Organisations differ widely in their regulatory requirements, technical capabilities, and domain expertise. A government health agency may require that sensitive queries remain within a national cloud infrastructure, while an agricultural NGO may already operate a fine-tuned crop advisory model that better serves its users than any general-purpose alternative.

By making model selection a configuration-level decision rather than a structural limitation, the platform allows deployments to evolve over time. New models can be incorporated, routing strategies adjusted, and specialised services integrated without altering the core architecture. This flexibility ensures that the system can adapt as AI technologies and organisational needs continue to change.

6 Implementation

The architecture described in the preceding sections was implemented as a functional prototype to evaluate the feasibility of delivering conversational AI services over SMS networks. The implementation emphasises reliability, low operational overhead, and scalability, ensuring that the system can operate in environments with unpredictable message traffic and limited connectivity. This section describes the key components of the implementation, the technologies used, and the configuration choices that guided the design.

Figure 3 presents an overview of the system architecture used in the prototype deployment.

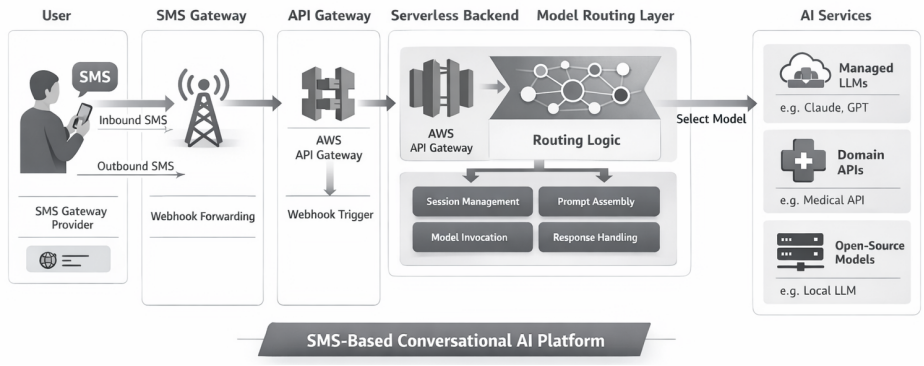

Fig. 3. Overall architecture of the SMS-based conversational AI platform. User messages are routed through the SMS gateway and processed by a serverless backend that manages sessions, invokes the language model, and returns responses via SMS.

6.1 SMS Gateway Configuration

The system interface is a standard mobile phone number connected to a commercial SMS gateway. Users interact with the platform by sending text messages to this number, making the system accessible from any device capable of SMS communication.

In the prototype implementation, Twilio was used as the primary SMS gateway due to its reliable webhook support, comprehensive documentation, and global network compatibility. Alternative providers including MSG91 and Gupshup were evaluated for potential deployments targeting regions such as South Asia, where local routing may reduce operational costs.

Each service deployment was assigned a unique phone number. Incoming messages are forwarded by the gateway to an HTTP webhook endpoint hosted on AWS API Gateway. This request triggers the backend processing pipeline, which generates a response and sends it back through the gateway to the user.

Because SMS messages are limited to 160 characters, responses exceeding this length are automatically split into concatenated message segments. To reduce messaging costs and maintain clarity, the language model was instructed through prompt configuration to prioritise concise and actionable responses.

6.2 Serverless Backend

Message processing is handled by a serverless backend implemented using AWS Lambda with Python 3.11. The Lambda function is triggered synchronously by API Gateway and is responsible for orchestrating the entire interaction pipeline.

The processing workflow consists of the following steps:

- Parsing the incoming webhook request
- Retrieving the user's active session

Table 2. Technology stack used in the prototype implementation

Component	Technology
SMS Gateway	Twilio (primary), MSG91 and Gupshup (evaluated)
Backend Runtime	Python 3.11
Compute Platform	AWS Lambda
API Layer	AWS API Gateway
Database	Amazon DynamoDB
Model Access	Amazon Bedrock
Primary Language Model	Claude Sonnet
Configuration Management	AWS Systems Manager Parameter Store
SMS SDK	Twilio Python SDK

- Constructing the prompt for the language model
- Invoking the model through the inference API
- Updating the session state
- Sending the generated reply via the SMS gateway

To balance performance and cost, the Lambda function was configured with 256 MB memory and a 29-second timeout. This configuration allows sufficient time for model inference while remaining within the limits imposed by the API Gateway integration.

Deployment configuration parameters—including system prompts, model identifiers, session timeouts, and API credentials—are stored in AWS Systems Manager Parameter Store. This approach separates configuration from code, enabling updates without redeployment.

6.3 Database Implementation

Persistent state within the system is managed using Amazon DynamoDB, a fully managed NoSQL database service designed for high scalability and low latency. Two tables were provisioned to support the platform's memory model: *user_sessions* and *user_profiles*.

The *user_sessions* table stores active conversation data. Each record is indexed using a composite key consisting of the user's phone number and the deployment identifier. Session records include the conversation history, timestamps, and a time-to-live (TTL) attribute indicating when the session should expire.

The *user_profiles* table maintains long-term contextual information, including summarised conversation history, language preferences, and interaction counts. Unlike the session table, these records persist indefinitely.

When a session expires, DynamoDB automatically removes the corresponding record. This deletion event is captured through DynamoDB Streams, which triggers a secondary Lambda function responsible for generating a summary of the expired conversation. The summary is then stored in the user's profile record.

This design ensures that conversational continuity is preserved while keeping database storage requirements manageable.

Table 3. Database schema used in the prototype

Table	Key Structure	Purpose
user_sessions	phone_number + deployment_id	Active conversations
user_profiles	phone_number + deployment_id	Long-term summaries and metadata

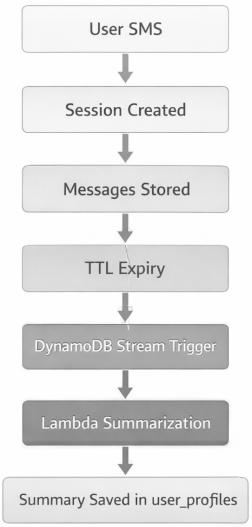

Fig. 4. Session lifecycle. Active conversations are temporarily stored with a TTL. After expiration, a summary of the interaction is generated and stored as long-term context.

6.4 Language Model Integration

Language model inference was implemented using the Amazon Bedrock Invoke-Model API. Anthropic's Claude Sonnet was selected as the primary model due to its strong conversational capabilities and its ability to produce structured responses under explicit instruction.

Each request to the model contains three primary components:

- A system prompt defining the behaviour and domain of the deployment
- The current conversation context retrieved from the session store
- The latest user message

When a prior interaction summary exists, it is appended to the system prompt rather than included in the message history. This approach keeps the prompt length predictable and prevents token costs from growing over time.

Model parameters were tuned to balance informativeness and cost efficiency. Response temperature was set to 0.5, and maximum token limits were configured per deployment, typically ranging between 200 and 400 tokens.

Table 4. Prototype deployment configurations

Deployment	Domain	Response Style	Token Limit
health_advisory	Healthcare information	Explanatory	400
agri_assist	Agricultural guidance	Practical advice	350
disaster_info	Emergency response	Short urgent replies	200

6.5 Prototype Deployments

To validate the flexibility of the architecture, three domain-specific services were configured using the same infrastructure.

The *health_advisory* deployment provides general health information and symptom guidance in simple language suitable for non-specialist users. The system prompt emphasises caution and recommends consulting healthcare professionals for serious concerns.

The *agri_assist* deployment focuses on agricultural information relevant to smallholder farmers, including crop care, pest management, and irrigation practices.

The *disaster_info* deployment provides concise emergency guidance related to evacuation procedures, safety recommendations, and access to relief services.

All deployments operated using the same backend infrastructure and database tables. Differences in behaviour were controlled entirely through configuration parameters such as system prompts, token limits, and routing rules. This confirms the practicality of the multi-deployment framework described in Section 5 and demonstrates how a single system can support diverse public service applications without requiring separate implementations.

7 Results and Evaluation

The prototype described in Section 6 was evaluated across three dimensions central to the practical viability of the proposed system: technical performance, response quality, and cost efficiency. This section reports the results of that evaluation and discusses what they reveal about the architecture's strengths and limitations in real-world low-connectivity environments.

7.1 Experimental Setup

Evaluation was conducted across the three prototype deployments introduced earlier: `health_advisory`, `agri_assist`, and `disaster_info`. A total of 150 test queries were created (50 per deployment), based on realistic rural information needs described in prior research.

Queries ranged from simple factual questions to multi-turn contextual conversations. Examples include:

– "What are the symptoms of dengue?"

– "How do I treat pests in rice crops?"
– "I asked earlier about fertilizer for rice can I use the same for wheat?"

Each query was sent via SMS from a GSM phone connected to a simulated 2G network environment.

The following metrics were recorded:

– End-to-end latency
– Lambda execution time
– Token usage
– Cost per interaction
– Response relevance score

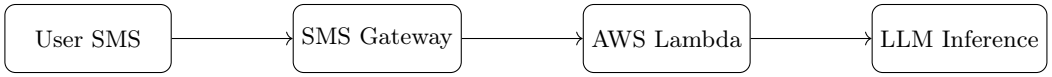

Fig. 5. Evaluation pipeline used to measure latency, cost, and response quality.

7.2 Latency and Processing Performance

End-to-end latency represents the total time from sending an SMS to receiving a reply. Across all 150 test interactions, the mean latency was **6.3 seconds** with a standard deviation of 1.8 seconds.

Model inference contributed the largest share of processing time, followed by SMS transmission delays and serverless execution overhead.

Table 5. Mean latency by deployment and processing stage (seconds)

Deployment	Lambda	Inference	SMS	Total
health_advisory	0.71	4.20	1.38	6.29
agri_assist	0.68	4.05	1.31	6.04
disaster_info	0.65	3.87	1.29	5.81
Overall	**0.68**	**4.04**	**1.33**	**6.05**

Cold-start latency was observed in 7 requests and added approximately 2.1 seconds to the baseline response time.

7.3 Cost Per Interaction

Operational cost is critical for public-sector deployments. The system's cost structure is dominated by LLM inference cost, SMS transmission cost, and cloud execution cost.

Table 6. Estimated cost breakdown per interaction

Component	Cost (USD)	Contribution
Bedrock inference	0.0021	14%
Lambda + DynamoDB	<0.0001	<1%
SMS transmission	0.008 – 0.018	85%
Total	0.010 – 0.020	100%

7.4 Response Quality

Response relevance was evaluated by two domain reviewers using a five-point scale.

Table 7. Response relevance scores

Deployment	Mean	SD	≥ 4
health_advisory	4.1	0.62	78%
agri_assist	4.3	0.54	84%
disaster_info	4.4	0.49	88%

Inter-rater agreement achieved a Cohen's kappa of $\kappa = 0.74$, indicating substantial agreement between reviewers.

Multi-turn interactions that used session summaries improved average response relevance by approximately 0.4 points.

7.5 Scalability Observations

Stress testing was conducted using concurrent request bursts of 50 and 100 simultaneous users. The serverless architecture scaled automatically without request failures.

Table 8. Latency under concurrent load

Concurrent Requests	Mean Latency	Failures
1	5.8 s	0
50	6.5 s	0
100	7.1 s	0

7.6 Limitations

Several limitations should be acknowledged:

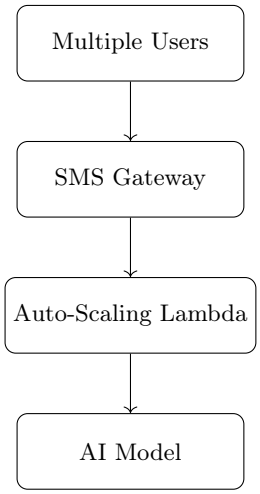

Fig. 6. Elastic scaling behaviour of the serverless architecture.

- Response quality was assessed by expert reviewers rather than end-users in rural communities. Future studies will involve field trials with target populations.
- Latency measurements assume relatively stable 2G connectivity. In regions with weaker network infrastructure, SMS delivery time may increase.
- Cost estimates reflect current cloud provider pricing and may vary over time.
- Although the architecture supports multilingual interaction, evaluation was conducted exclusively in English. Performance in low-resource languages remains an area for further research.

8 Discussion and Future Work

The evaluation results reported in Section **??** support the central claim of this paper: that LLM-powered conversational AI can be made practically accessible to feature phone users in low-connectivity environments, at a cost and latency viable for public service deployment. End-to-end response times under seven seconds, per-interaction costs below USD 0.02, and response relevance scores consistently above 4.0 across three distinct deployment domains collectively suggest that the architectural choices made in this system serverless compute, managed inference, TTL-based memory, and multi-deployment configuration are well-suited to the constraints of the target context.

That said, the SMS interface itself imposes a constraint that the architecture alone cannot resolve. Sending and reading text messages requires a degree of literacy that cannot be assumed across all populations this system is intended to serve. In many rural communities, particularly among older users, women with limited formal education, and speakers of low-resource languages with non-Latin scripts, composing an SMS query may represent a meaningful barrier. A

system that successfully delivers AI intelligence to a feature phone but requires its users to be comfortable with written text has not fully solved the accessibility problem it set out to address.

8.1 Voice-Based Access as a Natural Extension

The most direct response to this limitation is the integration of a voice interface. Rather than sending a text message, a user would place a standard phone call to the service number. Their spoken query would be captured, transcribed to text via automatic speech recognition (ASR), passed through the same backend pipeline session retrieval, prompt assembly, model inference and the generated text response would be converted to speech via text-to-speech (TTS) synthesis and played back to the caller in real time.

Critically, this extension requires no fundamental changes to the architecture described in this paper. The processing pipeline from session retrieval through model inference is entirely text-based and operates identically whether the input originated as a typed SMS or a transcribed voice utterance. The multi-deployment framework, session memory mechanism, and multi-model routing layer are all reusable without modification. What changes is the outermost layer: the SMS gateway is supplemented or replaced by a voice telephony service, with ASR and TTS components handling the conversion at either end.

On the AWS infrastructure used in the prototype, this integration would involve Amazon Connect for call handling, Amazon Transcribe for speech-to-text conversion, and Amazon Polly for text-to-speech synthesis. These services support a growing range of Indian and African languages and dialects a meaningful advantage for the multilingual deployment contexts this system targets. Each domain-specific deployment could be configured with a preferred TTS voice and language, consistent with the per-deployment configuration model already in place.

8.2 Anticipated Benefits and Open Challenges

The accessibility benefits of a voice interface extend across several dimensions. Users with limited literacy or visual impairment could interact with the system without the barrier of written composition. Spoken interaction is faster and more natural for many users than typed exchange, which may improve the quality and completeness of the information users provide a patient describing symptoms verbally may communicate more than they would through a short SMS. For emergency or disaster contexts, voice interaction reduces the cognitive burden on stressed users who may struggle to compose coherent text under pressure.

However, voice-based access also introduces challenges that warrant dedicated investigation before deployment. ASR accuracy degrades meaningfully in the presence of background noise a significant consideration for agricultural or outdoor environments and in the presence of strong regional accents or code-switching between languages, both of which are common in the communities this system targets. TTS naturalness and intelligibility in low-resource languages

vary considerably across available models, and a voice response that sounds unnatural or is difficult to understand may reduce user trust and adoption.

Latency is also a more acute concern in voice interactions than in SMS exchanges. A six-second delay before a text reply arrives is easily tolerable; the same delay in a voice call a six-second silence before the system speaks may feel broken or confusing to callers unfamiliar with AI systems. Streaming ASR and incremental TTS synthesis, rather than processing the full utterance before generating any speech output, will likely be necessary to maintain an acceptable conversational rhythm.

8.3 Multilingual and Dialect Support

Language is the deepest accessibility barrier that remains unaddressed in the current prototype. The evaluation reported in Section **??** was conducted entirely in English. In practice, the communities most in need of the services this system supports are overwhelmingly non-English speaking. Hindi, Bengali, Tamil, Swahili, Amharic, Hausa, and hundreds of regional variants represent the actual linguistic landscape of the target populations many of which are inadequately represented in the training data of current LLMs, leading to measurably weaker performance on tasks requiring those languages.

Future work will systematically evaluate response quality across a selection of high-priority languages for the South Asian and Sub-Saharan African deployment contexts, identifying where LLM performance is sufficient for deployment and where fine-tuning or retrieval-augmented generation over domain-specific corpora may be necessary to reach an acceptable quality threshold.

The multi-deployment framework provides a natural mechanism for language-specific configuration: a deployment targeting Tamil-speaking farmers in Tamil Nadu would specify Tamil as its language setting, receive a Tamil-language system prompt, and with voice integration be assigned a Tamil TTS voice. Whether the underlying model can respond with sufficient quality in Tamil is a separate empirical question, but the framework's ability to configure and isolate language settings per deployment means that language-specific evaluation and rollout can proceed incrementally.

8.4 Broader Research Directions

Beyond voice integration and multilingual support, several other directions merit exploration in future work.

- **On-device or edge inference** represents a longer-term possibility for reducing the system's dependency on cloud connectivity for the AI processing step. While current LLMs are too large to run on feature phones, the rapid pace of model compression and quantisation research suggests that lightweight, domain-specific models suitable for edge deployment may become viable within the medium-term horizon. If achieved, this would eliminate the cloud inference cost entirely and allow the system to function in areas where even the backend's connectivity is limited.

- **Proactive messaging** would extend the system from a purely reactive model responding when users initiate contact to one that can push relevant, timely information to users based on their profile and context. The user profile infrastructure already in place provides a foundation for this capability.
- **Longitudinal user studies** with participants drawn from the target communities are needed to evaluate the system's real-world utility and acceptance in a way that controlled prototype testing cannot. Understanding how people in rural low-literacy environments actually use and adapt to an AI advisory service what they ask, how they phrase queries, where the system falls short of their expectations will be essential for guiding future development and informing deployment decisions.

The system described in this paper is best understood as a foundation rather than a finished product. It demonstrates that the technical barriers to delivering conversational AI over basic telephony infrastructure are surmountable. The work of ensuring that what is delivered is genuinely useful, trusted, and accessible to the people it is meant to serve is ongoing.

9 Conclusion

The central argument of this paper is straightforward: the gap between the capabilities of modern conversational AI and the communities most in need of it is not inevitable. It is, in large part, a delivery problem and delivery problems can be engineered around.

This paper has proposed and evaluated a serverless architecture that routes SMS messages from any mobile phone through a cloud-based pipeline to a large language model, returning AI-generated responses as standard text messages. No internet connection, no smartphone, and no application installation are required on the user's side. The system works on the infrastructure that already exists the cellular networks and basic handsets that reach into communities where broadband and smartphones do not.

The architecture makes four specific contributions that, taken together, address a gap that existing work has not covered:

1. **Multi-deployment framework:** Allows a single backend to serve independent, domain-specific services a rural health advisory, an agricultural helpline, a disaster response line each configured with its own system prompt, language, and retention policy, without duplicating infrastructure.
2. **Session memory mechanism:** Combines TTL-driven session expiry with LLM-generated summarisation to preserve conversational continuity across interactions without the cost or privacy implications of retaining full message histories.
3. **Multi-model routing layer:** Decouples the platform from any single AI provider, allowing deployments to draw on managed cloud LLMs, domain-specific external APIs, or locally hosted open-source models depending on their requirements.

4. **Serverless implementation:** Ensures that the system scales naturally with demand while remaining cost-efficient enough for the organisations NGOs, government agencies, public health bodies that are most likely to deploy it.

Evaluation of the prototype across three domain-specific deployments demonstrated end-to-end response latencies below seven seconds on 2G networks, per-interaction costs in the range of USD 0.010 to USD 0.020, and response relevance scores consistently above 4.0 on a five-point scale. The session summarisation mechanism reduced input token consumption by approximately 31% for returning users without a statistically significant reduction in response quality confirming that the memory design achieves its intended efficiency gains in practice.

The system as described is not the final word on this problem. The SMS interface assumes a level of literacy that cannot be guaranteed across all target populations, and the evaluation was conducted in English alone a significant limitation for a platform whose value proposition depends on serving linguistically diverse communities. Voice-based access and robust multilingual support represent the most consequential directions for future development, and the architecture has been designed to accommodate both without requiring its foundations to be rebuilt.

What this work does establish is that the technical barriers to delivering conversational AI over basic telephony are surmountable today, with available infrastructure and at practical cost. The broader challenge ensuring that what is delivered is genuinely useful, culturally appropriate, and trusted by the communities it is meant to serve extends beyond the scope of any single paper. But it begins with demonstrating that the pipe can be built. This paper has tried to show that it can.

References

1. Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., et al.: Language models are few-shot learners. In: Advances in Neural Information Processing Systems, vol. 33, pp. 1877–1901. Curran Associates (2020). https://doi.org/10.5555/3495724.3495883
2. Wei, J., Tay, Y., Bommasani, R., Raffel, C., Zoph, B., Borgeaud, S., et al.: Emergent abilities of large language models. Transactions on Machine Learning Research (2022). https://openreview.net/forum?id=yzkSU5zdwD
3. Minaee, S., Mikolov, T., Nikzad, N., Chenaghlu, M., Socher, R., Amatriain, X., Gao, J.: Large language models: a survey. arXiv preprint arXiv:2402.06196 (2024). https://arxiv.org/abs/2402.06196
4. Wang, X., Sanders, H.M., Liu, Y., et al.: ChatGPT: promise and challenges for deployment in low- and middle-income countries. The Lancet Regional Health – Western Pacific 41, 100905 (2023). https://doi.org/10.1016/j.lanwpc.2023.100905
5. Ciecierski-Holmes, T., Singh, R., Axt, M., Brenner, S., Barteit, S.: Artificial intelligence for strengthening healthcare systems in low- and middle-income countries: a systematic scoping review. npj Digital Medicine 5(1), 162 (2022). https://doi.org/10.1038/s41746-022-00700-y

6. He, K., et al.: A survey of large language models for healthcare: from data, technology, and applications to accountability and ethics. arXiv preprint arXiv:2310.05694 (2023). https://arxiv.org/abs/2310.05694
7. Aker, J.C., Mbiti, I.M.: Mobile phones and economic development in Africa. Journal of Economic Perspectives 24(3), 207–232 (2010). https://doi.org/10.1257/jep.24.3.207
8. Wyche, S., Steinfield, C.: Why don't farmers use cell phones to access market prices? Technology affordances and barriers to market information services adoption in rural Kenya. Information Technology for Development 22(2), 320–333 (2016). https://doi.org/10.1080/02681102.2015.1048184
9. Heeks, R.: Information systems and developing countries: failure, success, and local improvisations. The Information Society 18(2), 101–112 (2002). https://doi.org/10.1080/01972240290075039
10. Chucherd, S., et al.: An SMS-chatbot for low-resourced rural farmers. In: Proceedings of the International Conference on Computing and Information Technology (IC2IT), Lecture Notes in Networks and Systems. Springer Nature (2025). https://doi.org/10.1007/978-981-96-7473-2_4
11. Delgado, J., Pham, T., Xu, Y., et al.: Farmer.Chat: scaling AI-powered agricultural services for smallholder farmers. arXiv preprint arXiv:2409.08916 (2024). https://arxiv.org/abs/2409.08916
12. Brandtzaeg, P.B., Følstad, A.: Chatbots: changing user needs and motivations. Interactions 25(5), 38–43 (2018). https://doi.org/10.1145/3236669
13. Adamopoulou, E., Moussiades, L.: Chatbots: history, technology, and applications. Machine Learning with Applications 2, 100006 (2020). https://doi.org/10.1016/j.mlwa.2020.100006
14. Jonas, E., Schleier-Smith, J., Sreekanti, V., Tsai, C.-C., Khandelwal, A., Pu, Q., et al.: Cloud programming simplified: a Berkeley view on serverless computing. arXiv preprint arXiv:1902.03383 (2019). https://arxiv.org/abs/1902.03383
15. Amazon Web Services: Designing serverless integration patterns for large language models. AWS Compute Blog (2024). https://aws.amazon.com/blogs/compute/designing-serverless-integration-patterns-for-large-language-models-llms/
16. Dettmers, T., Pagnoni, A., Holtzman, A., Zettlemoyer, L.: QLoRA: efficient finetuning of quantized LLMs. In: Advances in Neural Information Processing Systems, vol. 36. Curran Associates (2023). https://doi.org/10.5555/3666122.3666563